

//Example for vector

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> numbers;

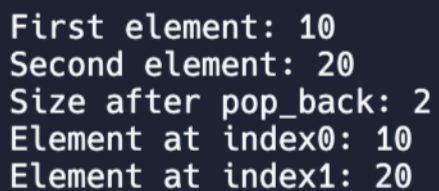
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    cout << "First element: " << numbers[0] << endl;
    cout << "Second element: " << numbers[1] << endl;

    numbers.pop_back();

    cout << "Size after pop_back: " << numbers.size() << endl;

    for (int i = 0; i < numbers.size(); i++){
        cout << "Element at index" << i << ": " << numbers[i]<< endl;
    }
    return 0;
}
```



```
First element: 10
Second element: 20
Size after pop_back: 2
Element at index0: 10
Element at index1: 20
```

//Iterators

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> vec = {1, 2, 3, 4, 5};
    for (auto it = vec.begin(); it != vec.end(); it++) {
        cout << *it << endl;
        //for (vector<int>::iterator it = vec.begin(); it != vec.end(); it++) {
        //cout << *it << endl;
        //}
    }
```

```
}  
return 0;  
}
```

```
1  
2  
3  
4  
5
```

//Smart pointers in C++ (examples 1-4)

```
#include <iostream>  
#include <memory>  
using namespace std;  
int main() {  
    unique_ptr<int> p1 = make_unique<int>(10);  
    cout << "Value inside unique_ptr: " << *p1 << endl;  
    return 0;  
}
```

```
Value inside unique_ptr: 10
```

```
#include <iostream>  
#include <memory>  
using namespace std;  
int main() {  
    shared_ptr<int> p1 = make_shared<int>(10);  
    shared_ptr<int> p2 = p1;  
    cout << "Value inside shared_ptr p1: " << *p1 << endl;  
    cout << "Value inside shared_ptr p2: " << *p2 << endl;  
    return 0;  
}
```

```
Value inside shared_ptr p1: 10  
Value inside shared_ptr p2: 10
```

```
#include <iostream>
```

```
#include <memory>
using namespace std;
int main() {
    unique_ptr<int> p1 = make_unique<int>(10);
    cout << "Value (the memory address) of p1 = " << p1 << endl;
    cout << "Value inside unique_ptr: " << *p1 << endl;
    return 0;
}
```

```
./main.cpp:6:51: error: invalid operands to binary expression ('basic_ostream<char, char_traits<char>>' and
'unique_ptr<int>')
6 |         cout << "Value (the memory address) of p1 = " << p1 << endl;
  |         ~~~~~^~~~~~
```

(This code could be fixed by adding * in front of p1 to point to the memory address.)

```
#include <iostream>
#include <memory>
using namespace std;
int main() {
    unique_ptr<int> p1 = make_unique<int>(10);
    cout << "Value (the memory address) of p1 = " << p1.get() << endl;
    cout << "Value inside unique_ptr: " << *p1 << endl;
    return 0;
}
```

```
Value (the memory address) of p1 = 0x55fdbcfb72b0
Value inside unique_ptr: 10
```

//Basic Lambda Example

```
#include <iostream>
using namespace std;

int main(){
    auto sum = [](int a, int b){
        return a + b;
    };
    cout << "Sum of 3 and 4 is " << sum(3, 4) << endl;
    return 0;
}
```

```
Sum of 3 and 4 is 7
```

//Capturing variables in c++

```
#include <iostream>
```

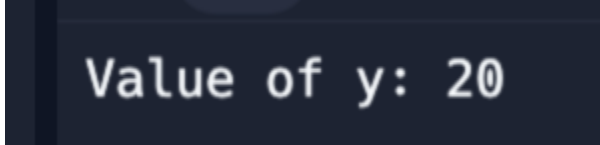
```
using namespace std;
int main() {
    int x = 10;
    auto printX = [x]() {
        cout << "Captured x: " << x << endl;
    };
    printX();
    return 0;
}
```



Captured x: 10

//compact syntax, ternary operator

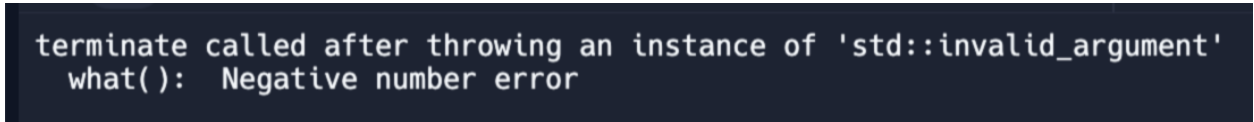
```
#include <iostream>
using namespace std;
int main() {
    int x = 10;
    int y = (x > 5) ? 20 : 30;
    cout << "Value of y: " << y << endl;
    return 0;
}
```



Value of y: 20

//error checking and exception (example 1 and 2)

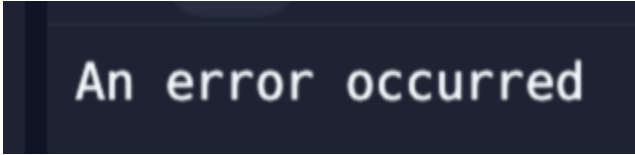
```
#include <iostream>
#include <stdexcept>
using namespace std;
int main() {
    int x = -1;
    if (x < 0) {
        throw invalid_argument("Negative number error");
    }
    return 0;
}
```



```
terminate called after throwing an instance of 'std::invalid_argument'
what():  Negative number error
```

```
#include <iostream>
```

```
#include <stdexcept>
using namespace std;
int main() {
    try {
        throw runtime_error("An error occurred");
    } catch (const exception& e) {
        cout << e.what() << endl;
    }
    return 0;
}
```



An error occurred